

A Comparative Analysis of Custom Kernel Support Vector Machines with Optimized Spread-Based Metrics

Eduardo Henrique Basilio de Carvalho
Departamento de Engenharia Eletrônica
Universidade Federal de Minas Gerais
Belo Horizonte, Brasil
eduardohbc@ufmg.br

Abstract—This paper presents a comparative study of custom kernel Support Vector Machines (SVMs) where kernel hyperparameters are optimized using novel objective metrics based on data spread characteristics. Two custom SVM configurations, one employing a spatial spread metric (M_{2D}) and another an axis-based spread metric (M_{Axis}), are evaluated against a standard SVM (M_{Ref}), all incorporating an identical comprehensive preprocessing pipeline. The preprocessing includes variance thresholding, correlation-based feature filtering, standardization, and Principal Component Analysis. The custom kernel's bandwidth parameter (h) is optimized by maximizing specific objective functions derived from either 2D spatial point spread or 1D vector spread measures. An experimental framework using 10-fold stratified cross-validation is applied across 20 datasets, measuring accuracy and computation time, with `float64` precision. Statistical comparisons using paired t-tests indicate that while the custom SVMs often achieve accuracy statistically equivalent to the standard SVM, they do not consistently offer superior accuracy and, in some instances, perform significantly worse. A notable observation is the significantly higher computational cost associated with the custom models due to their internal hyperparameter optimization process. The use of `float64` precision appeared to yield more stable or favorable results for the custom methods compared to indications from prior `float32` analyses.

Index Terms—Support Vector Machines, Custom Kernel, Kernel Optimization, Spread Metrics, Machine Learning, Classification, Performance Comparison.

I. INTRODUCTION

Support Vector Machines (SVMs) are powerful supervised learning models widely used for classification tasks due to their ability to find optimal separating hyperplanes in high-dimensional spaces. The choice of kernel function and its parameters is crucial for the performance of SVMs, as it defines the transformed feature space where the separation is performed. While standard kernels like the Radial Basis Function (RBF) are commonly used, the development and optimization of custom kernels or kernel hyperparameters tailored to specific data characteristics or objectives can potentially enhance performance.

This paper introduces two custom SVM configurations where a key kernel hyperparameter, h (bandwidth), is optimized based on objective metrics derived from data spread

characteristics. These metrics aim to quantify class separability and cohesion in a transformed space influenced by the kernel matrix itself. The first custom SVM (M_{2D}) utilizes an objective function based on 2D spatial spread distances, while the second (M_{Axis}) employs an objective function based on 1D vector spread measures.

To provide a comprehensive evaluation, these custom SVMs are integrated into pipelines with a standardized, multi-stage preprocessing procedure. Their performance, in terms of classification accuracy and computation time, is rigorously compared against a standard SVM pipeline using the same preprocessing, across a diverse set of 20 benchmark datasets. This study primarily reports results obtained using `float64` precision, which has shown some improvements in the stability and performance of the custom methods over preliminary `float32` analyses.

The remainder of this paper is structured as follows: Section II details the methodology, including the preprocessing pipeline, the kernel definition and initial parameter estimation, the architecture of the custom kernel SVM, and the specific objective functions used for hyperparameter optimization. Section III describes the experimental setup for the comparative analysis. Section IV presents and discusses the results from these experiments. Finally, Section V concludes the paper and suggests avenues for future research.

II. METHODOLOGY

The proposed approach involves several key components: a data preprocessing pipeline, a specific parameterized kernel function with its initial parameter estimation, custom SVM classifiers that optimize a kernel hyperparameter (h) using novel objective functions, and these objective functions themselves.

A. Preprocessing Pipeline

A sequential preprocessing transformer is applied to the data before being fed to the SVM classifiers. This pipeline ensures data consistency and aims to improve model performance by addressing common data quality issues. The stages are applied in the following order:

- 1) **Low-Variance Feature Removal (T_1):** Features with a sample variance $s_j^2 = \frac{1}{N-1} \sum_{k=1}^N (X_{kj}^{(0)} - \bar{X}_j^{(0)})^2$ below a threshold $\tau_{var} = 0.05$ are removed.
- 2) **High-Correlation Feature Filtering (T_2):** This stage removes features that are highly correlated with others. A Pearson correlation matrix $\mathbf{C}$ is computed, and if for any pair of features (i, j) with $i < j$, $|C_{ij}| > \tau_{corr}$ (where $\tau_{corr} = 0.95$), feature j is dropped.
- 3) **Data Scaling (Standardization) (T_3):** Features are standardized by removing the mean and scaling to unit variance. For each feature value $X_{kj}^{(2)}$, the transformed value is $X_{kj}^{(3)} = (X_{kj}^{(2)} - \bar{X}_j^{(2)})/s_j^{(2)}$.
- 4) **Principal Component Analysis (PCA) (T_4):** PCA is applied for dimensionality reduction, transforming the data into a new set of orthogonal features that capture maximum variance. The data $\mathbf{X}^{(3)}$ is projected onto D_{pca} principal components: $\mathbf{X}^{(4)} = \mathbf{X}^{(3)}\mathbf{W}$.

B. Kernel Definition and Initial Parameter Estimation

The custom SVMs utilize a specific parameterized kernel function whose initial parameters are estimated via an "Initial Kernel Parameter Estimation Procedure".

1) **Initial Parameter Estimation:** This procedure estimates a normalization factor $\eta \in \mathbb{R}$ and an inverse matrix $\Psi^{-1} \in \mathbb{R}^{D \times D}$ from the input data matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$. Two estimation types are supported:

- **'cov' type:** Ψ^{-1} is the inverse of the empirical covariance matrix Σ of $\mathbf{X}$, and $\eta = 1/((2\pi)^{D/2} \sqrt{\det(\Sigma)})$. This requires validation of Σ for non-singularity.
- **'scale' type:** Ψ^{-1} is the identity matrix $\mathbf{I}$, and $\eta = 1/((2\pi)^{D/2})$. The experiments in this paper use the 'scale' type for `kernel_fit_type`.

2) **Parameterized Kernel Function ('kernel'):** This function computes an $N_X \times N_Y$ kernel matrix $\mathbf{K}$ between two data matrices $\mathbf{X} \in \mathbb{R}^{N_X \times D}$ and $\mathbf{Y} \in \mathbb{R}^{N_Y \times D}$, using the initial parameters η_0, Ψ_0^{-1} and a bandwidth hyperparameter $h \neq 0$. The steps are:

- 1) **Parameter Scaling:**
 - Scaled Inverse Matrix: $\Psi_h^{-1} = \Psi_0^{-1}/h^2$.
 - Scaled Normalization Factor: $\eta_h = \eta_0/h^D$ (with boundary conditions for small h^D or $\eta_0 = 0$).
- 2) **Quadratic Form Matrix (Q):** Elements $Q_{ij} = (\mathbf{x}_i - \mathbf{y}_j)^T \Psi_h^{-1} (\mathbf{x}_i - \mathbf{y}_j)$ are computed.
- 3) **Unnormalized Kernel-like Values (P):** $P_{ij} = \eta_h \exp(-\frac{1}{2}Q_{ij})$.
- 4) **Final Normalization:** $K_{ij} = P_{ij}/\max(\mathbf{P})$, or 0 if $\max(\mathbf{P}) \approx 0$.

C. Custom Kernel Support Vector Classifier ('CustomKernelSVC')

The Custom Kernel SVM ('CustomKernelSVC') is designed to optimize the kernel hyperparameter h using a specified objective metric.

a) **Initialization:** Key parameters include bounds for h optimization (h_{bounds}), the type for `kernel_fit`, the choice of `objective_metric` ('spatial' or 'axis'), and arguments for the underlying 'sklearn.svm.SVC'.

b) **Training Procedure:**

- 1) Input validation and storage of training data $\mathbf{X}_{train}$ and unique classes $\mathcal{C}$.
- 2) Initial kernel parameters (η, Σ^{-1}) are estimated using `kernel_fit( $\mathbf{X}_{train}$ , type = kernel_fit_type)`. (Note: Σ^{-1} is a general notation for Ψ_0^{-1} here).
- 3) The hyperparameter h is optimized to h_{opt} by maximizing a selected score $S(h)$ within h_{bounds} , using a scalar minimization routine on $-S(h)$. $S(h)$ is determined by the "Hyperparameter Optimization Objective Criterion".
- 4) The final training kernel matrix $\mathbf{K}_{train_opt} = \text{kernel}(\mathbf{X}_{train}, \mathbf{X}_{train}, \eta, \Sigma^{-1}, h_{opt})$ is computed.
- 5) An 'SVC(kernel='precomputed')' is trained on $\mathbf{K}_{train_opt}$ and labels $\mathbf{y}$.

c) **Hyperparameter Optimization Objective Criterion:**

This criterion is minimized to find h_{opt} . For a candidate h :

- 1) The kernel matrix $\mathbf{K}_h = \text{kernel}(\mathbf{X}_{train}, \mathbf{X}_{train}, \eta, \Sigma^{-1}, h)$ is computed.
- 2) Projection vectors $\mathbf{Q}_{C_0}(h)$ and $\mathbf{Q}_{C_1}(h)$ are formed. For each sample $\mathbf{x}_i$: $(\mathbf{Q}_{C_k}(h))_i = \sum_{j \text{ s.t. } y_j = C_k} (\mathbf{K}_h)_{i,j}$, where C_k are the first two unique classes.
- 3) The score $S(h)$ is calculated using either $S_{spatial}(\mathbf{Q}_{C_0}(h), \mathbf{Q}_{C_1}(h), \mathbf{y})$ or $S_{axis}(\mathbf{Q}_{C_0}(h), \mathbf{Q}_{C_1}(h), \mathbf{y})$, based on `objective_metric`.
- 4) The criterion returns $-S(h)$ (or ∞ if $S(h)$ is NaN).

d) **Prediction Procedure ('predict'):** For new samples $\mathbf{X}_{new}$, the kernel matrix $\mathbf{K}_{test} = \text{kernel}(\mathbf{X}_{new}, \mathbf{X}_{train}, \eta, \Sigma^{-1}, h_{opt})$ is computed, and predictions are made by the trained SVC using $\mathbf{K}_{test}$.

D. Objective Functions for Hyperparameter Optimization

Two distinct objective functions are employed, chosen by the `objective_metric` parameter.

1) **Axis-Spread Objective Function (S_{axis}):** This function evaluates the evenness of data distribution within two feature sets derived from kernel projections, conditioned on class labels.

a) **Inputs:** It takes a primary numerical sequence X , a secondary numerical sequence W (here, $\mathbf{Q}_{C_0}(h)$ and $\mathbf{Q}_{C_1}(h)$ respectively), and binary class labels L (here, $\mathbf{y}$).

b) **Procedure:**

- 1) Sub-sequences $V_{X_0} = \{x_i \mid l_i = 0\}$ and $V_{W_1} = \{w_i \mid l_i = 1\}$ are extracted.
- 2) A "Vector Evenness Score" S_V is calculated for each sub-sequence. This score itself is complex:
 - For a sequence $V = (v_1, \dots, v_n)$, it's 1.0 if $n = 1$; undefined if $n = 0$ or has NaNs.
 - For $n \geq 2$: sort V to V'_{sorted} . Compute differences $\Delta = (\delta_j)$, where $\delta_j = v'_{(j+1)} - v'_{(j)}$.

- Calculate mean μ_Δ , population standard deviation σ_Δ , and max δ_{max} of Δ .
- Standard Deviation Score $S_\sigma = 1/(1 + \sigma_\Delta)$ (undefined if σ_Δ is NaN/Inf).
- Mean Spacing Score $S_\mu = (\mu_\Delta/\delta_{max})$ if $\delta_{max} \neq 0$, else 0.
- Final Vector Evenness Score $S_V = S_\sigma \times S_\mu$.

Let these scores for V_{X_0} and V_{W_1} be S_{X_0} and S_{W_1} .

- 3) The objective function value is $J_O = \frac{S_{X_0} + S_{W_1}}{2} - |S_{X_0} - S_{W_1}|$ (undefined if S_{X_0} or S_{W_1} is undefined).

2) *Spatial Spread Objective Function* ($S_{spatial}$): This function provides a single value balancing inter-group separation and intra-group cohesion for 2D point distributions. The inputs $Q_{C_0}(h)$ and $Q_{C_1}(h)$ are treated as coordinates of N points, used with labels y .

a) *Inputs*: A collection of M points (derived from $Q_{C_0}(h)$, $Q_{C_1}(h)$ treating them as coordinates) and M class labels L (here, y).

b) *Component Calculations*:

- 1) Mean Inter-Group Distance (M_{inter}): Average Euclidean distance between all pairs of points in opposite groups, derived from points in $G_0 = \{P_k \mid l_k = 0\}$ and $G_1 = \{P_k \mid l_k = 1\}$. Requires $N_0, N_1 > 0$.
- 2) Mean Intra-Group Distance for Group 0 ($M_{intra,0}$): Average Euclidean distance between unique pairs of points within G_0 . Requires $N_0 \geq 2$.
- 3) Mean Intra-Group Distance for Group 1 ($M_{intra,1}$): Similarly for G_1 . Requires $N_1 \geq 2$.
- 4) Average of Mean Intra-Group Distances: $\bar{M}_{intra} = (M_{intra,0} + M_{intra,1})/2$.
- 5) Population Standard Deviation of Mean Intra-Group Distances: $\sigma_{M_{intra}} = |M_{intra,0} - M_{intra,1}|/2$.

c) *Objective Function Value* (J): $J = \bar{M}_{intra} + \sigma_{M_{intra}}$, which simplifies to $J = M_{inter} + \min(M_{intra,0}, M_{intra,1})$. The value is undefined if any component cannot be computed.

III. EXPERIMENTAL SETUP

The performance of the custom SVM configurations (M_{2D} and M_{Axis}) was compared against a reference model (M_{Ref}) using a standardized experimental framework.

• Model Configurations:

- M_{2D} : "Sequential Preprocessing Transformer" + "Custom Kernel Support Vector Classifier" (objective_metric='spatial', kernel_fit_type='scale').
- M_{Axis} : "Sequential Preprocessing Transformer" + "Custom Kernel Support Vector Classifier" (objective_metric='axis', kernel_fit_type='scale').
- M_{Ref} : "Sequential Preprocessing Transformer" + Standard 'sklearn.svm.SVC'.

All SVMs used a fixed random state for reproducibility.

- **Datasets**: A total of 20 datasets were used for evaluation.
- **Cross-Validation**: Performance was assessed using $K_{cv} = 10$ fold stratified cross-validation.
- **Performance Metrics**:

- Classification Accuracy: Mean ($\bar{A}(M)$) and sample standard deviation ($s_A(M)$) over the K_{cv} folds were recorded.
- Computation Time: Total wall-clock time for the K_{cv} -fold cross-validation for each model.

- **Statistical Comparison**: Paired samples t-tests were performed on the K_{cv} accuracy scores between pairs of models. A p-value was computed for each comparison.
- **Equivalence Heuristic**: Two models were considered to have "statistically equivalent" performance if their t-test p-value was greater than $\alpha_{equiv} = 0.05$.
- **Precision**: The main results discussed are based on experiments conducted using float64 precision.

IV. RESULTS AND DISCUSSION

The analysis of experimental results using float64 precision for 19 datasets (excluding the 'adult' dataset due to errors with custom models) is presented below.

A. Accuracy Comparison

- M_{2D} vs. M_{Ref} : M_{2D} did not statistically significantly outperform M_{Ref} on any dataset. It was significantly worse on 3 datasets and performed equivalently on the remaining 16 datasets.
- M_{Axis} vs. M_{Ref} : M_{Axis} also did not significantly outperform M_{Ref} on any dataset. It was significantly worse on 2 datasets and equivalent on the remaining 17 datasets.
- M_{2D} vs. M_{Axis} : M_{2D} was significantly better than M_{Axis} on 3 datasets. M_{Axis} was significantly better than M_{2D} on 1 dataset ('sylvine'). They were equivalent on 15 datasets.

Overall, the custom SVMs, when operating with float64 precision, showed equivalent performance to the standard SVM more frequently than in prior float32 analyses. However, they did not demonstrate a consistent accuracy advantage and were sometimes outperformed by the reference model. The specific choice between 'spatial' (M_{2D}) and 'axis' (M_{Axis}) objective metrics did not yield a universally superior custom model, though M_{2D} had a slight edge in direct comparisons.

B. Computation Time Comparison

A consistent finding was the substantially higher computation time for M_{2D} and M_{Axis} compared to M_{Ref} .

- The custom models were often tens to hundreds of times slower than the reference model. For example, on the 'mushroom' dataset, M_{Axis} was approximately 184 times slower than M_{Ref} (756.65s vs 4.10s).
- Times between M_{2D} and M_{Axis} varied, with neither being consistently faster across all datasets. For instance, M_{2D} was faster on 'spambase' (89.90s vs 172.33s), while M_{Axis} was faster on 'sylvine' (5.78s vs 7.41s).

This increased time is primarily due to the iterative hyperparameter optimization process embedded within the custom SVMs' fitting procedure for each cross-validation fold.

C. Dataset-Specific Observations

- **Impact of Precision:** The `float64` results showed fewer instances of significant underperformance by custom models compared to previous `float32` results, particularly on datasets like `german_credit_g`, `spambase`, `digits_binary_0_vs_1`, and `digits_binary_5_vs_rest`, where they are now largely equivalent to the reference. This suggests `float64` precision enhances the stability or effectiveness of the custom kernel’s optimization process.
- **‘sylvine’ Dataset:** M_{2D} (0.5015 accuracy) performed exceptionally poorly, significantly worse than M_{Ref} (1.0000) and M_{Axis} (0.9531). M_{Axis} achieved high accuracy, equivalent to M_{Ref} .
- **‘adult’ Dataset:** Both M_{2D} and M_{Axis} failed to produce accuracy scores (“Error or no scores”), whereas M_{Ref} completed successfully. This issue persisted from `float32` to `float64` experiments, indicating a more fundamental problem with the custom models on this dataset.
- **‘mushroom’ Dataset:** M_{Axis} achieved equivalence with M_{Ref} (both near perfect), while M_{2D} was statistically slightly worse (0.9990 vs 1.0000), despite both custom models exhibiting very long runtimes.

V. CONCLUSION

This paper presented and evaluated two custom kernel SVM configurations, M_{2D} and M_{Axis} , which optimize a kernel hyperparameter based on spatial and axis-wise spread metrics, respectively. The experiments, conducted with `float64` precision, demonstrated that these custom models often achieve classification accuracy statistically equivalent to a standard SVM using the same comprehensive preprocessing pipeline. However, they did not provide a consistent improvement in accuracy and, in a few cases, performed significantly worse. A critical trade-off is the substantially increased computation time associated with the custom models due to their internal optimization routines.

The transition from `float32` to `float64` precision appeared to mitigate some of the severe underperformance issues previously observed for the custom models on certain datasets, leading to more instances of equivalence with the reference model. Nevertheless, the custom approaches did not emerge as generally superior. The ‘sylvine’ dataset highlighted a case where the ‘spatial’ objective metric (M_{2D}) led to poor performance, while the ‘axis’ metric (M_{Axis}) was effective. Conversely, M_{2D} showed a slight advantage over M_{Axis} in three other specific instances.

Future work could explore the reasons for the errors encountered by custom models on the ‘adult’ dataset and investigate the specific data characteristics that cause significant performance drops (e.g., on ‘sylvine’ for M_{2D}). Further refinement of the objective functions or the optimization strategy for the kernel hyperparameter might lead to more robust and consistently performing custom SVMs. Additionally, exploring

methods to reduce the computational burden of the hyperparameter optimization process would be essential for practical applicability.